Задача: Дано множество S целых чисел и целое неотрицательное целое число c. Нужно найти сколько существует пар неотрицательных целых чисел x и y, таких что $0 \leqslant x \leqslant y \leqslant c$ и что $x + y$ и $y - x$ не принадлежат S. S задано в отсортированном порядке. **Задача отличается от оригинальной тем, что числа из S не ограничены c.**

Решение:

Посчитаем количество таких пар x и y, что они лежат в нужных границах и:

1. $x + y$ принадлежит S (N^+),
2. $y - x$ принадлежит S (N^-),
3. оба $y \pm x$ принадлежат S ($N^\pm$).

Тогда ответ это: $\frac{(c+1)(c+2)}{2} - N^+ - N^- + N^\pm$ согласно принципу включения-исключения.

Для подсчёта N^+ сделаем следующее. Переберём число a из S. Какие x могут давать a в сумме? Они должны быть больше 0 и $a - c$ (иначе не хватит y, чтоб добить до суммы). С другой стороны они должны быть меньше c и $\lfloor \frac{a}{2} \rfloor$ (чтоб y был не меньше x). Все числа из полученного интервала задают x, а значит и y. Эту часть можно сделать за $\mathcal{O}(|S|)$.

Аналогично подсчитаем N^-. Тут для числа a будем перебирать y. Он может быть любым от a до c. Сложность такая же.

Наконец, рассмотрим $N^\pm$. Заметим, что $y \pm x$ одной чётности, поэтому задача разбивается на две одинаковые: одна для чётных чисел из S, другая для нечётных. Обозначим $a = y - x$, $b = y + x$. Зафиксируем a, найдём все b, такие что существуют x, y, которые удовлетворяют условию задачи. Во-первых, b должно быть не меньше a. Во-вторых, оно не может быть больше, чем $2c - a$ (иначе x или y должен быть больше c). Все числа, которые удовлетворяют этим условиям, подходят. Так как $2c - a$ убывает при росте a, то можно найти все подходящие b для всех a в порядке возрастания с помощью метода скользящего окна за $\mathcal{O}(|S|)$.

```
#include <bits/stdc++.h>

using namespace std;
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
const ld PI = acosl(-1.);

ll process(const vector<int>& a, ll c) {
  ll ans = 0;
  int right = static_cast<int>(a.size()) - 1;
  for (int left = 0; left < a.size(); ++left) {
    ll cur = a[left];
    ll max_value = c + c - cur;
    while (right >= 0 && a[right] > max_value) {
      --right;
    }
    ans += max(0, right - left + 1);
  }
```

```
  return ans;
}

ll process_plus(const vector<int>& a, ll c) {
  ll ans = 0;
  for (ll x : a) {
    ll left = max(0LL, x - c);
    ll right = min(x / 2, c);
    ans += max(0LL, right - left + 1);
  }
  return ans;
}

ll process_minus(const vector<int>& a, ll c) {
  ll ans = 0;
  for (ll x: a) {
    ll left = x;
    ll right = c;
    ans += max(0LL, right - left + 1);
  }
  return ans;
}

int main() {
  int T;
  scanf("%d", &T);

  while (T--) {
    int n, c;
    scanf("%d%d", &n, &c);
    vector<int> odds, evens, a;
    while (n--) {
      int x;
      scanf("%d", &x);
      if (x & 1) odds.push_back(x);
      else evens.push_back(x);
      a.push_back(x);
    }
    ll have_both_plus_and_minus = process(odds, c) + process(evens, c);
    ll have_plus = process_plus(a, c);
    ll have_minus = process_minus(a, c);
    ll all = static_cast<ll>(c + 1) * (c + 2) / 2;
    ll ans = all - have_plus - have_minus + have_both_plus_and_minus;
    cout << ans << '\n';
  }
```

```
    return 0;
}
```

Источник: https://codeforces.com/contest/1935/problem/D